Sprawozdanie – Projekt 3: Podstawy Gymnasium

Paweł Adamczyk i Dominika Bujnarowska

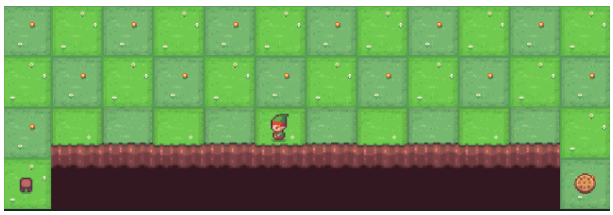
link do repozytorium: github

Część I
Środowisko dyskretne: CliffWalking

Algorytm: Q-Learning z tabelą wartości

1 Opis środowiska

Wybranym środowiskiem jest `CliffWalking-v1` z biblioteki Gymnasium. Jest to klasyczny problem planowania na siatce 4×12 pól, w którym agent porusza się w czterech kierunkach (góra, dół, lewo, prawo), próbując dotrzeć z lewego dolnego rogu siatki do prawego dolnego rogu, omijając krawędź urwiska.



Rysunek 1: Środowisko Cliff Walking

Parametr	**Wartość**
Przestrzeń stanów	Discrete(48) – 48 pól siatki 4×12
Przestrzeń akcji	Discrete(4) – góra, dół, lewo, prawo
Nagroda za krok	-1 pkt (każdy bezpieczny krok)
Nagroda za upadek	-100 pkt (teleportacja na start)
Wynik optymalny	-13 pkt (13 kroków bez upadku)

Środowisko zostało wybrane ze względu na w pełni dyskretny charakter przestrzeni stanów i akcji, co pozwala na bezpośrednie zastosowanie klasycznego algorytmu Q-Learning z tabelą wartości – bez konieczności dyskretyzacji.

2 Algorytm: Q-Learning

Q-Learning jest algorytmem uczenia ze wzmocnieniem typu *off-policy*, który uczy się optymalnej funkcji wartości $Q(s, a)$ — przypisującej każdej parze (stan, akcja) oczekiwaną sumę zdyskontowanych nagród.

Reguła aktualizacji ma postać równania Bellmana:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') \cdot (1 - \text{done}) - Q(s, a) \right]$$

Gdzie:

- $\alpha = 0.1$ – współczynnik uczenia (*learning rate*)
- $\gamma = 0.9$ – współczynnik dyskontowy (wymagany w projekcie)
- r – natychmiastowa nagroda ze środowiska
- $\max_{a'} Q(s', a')$ – najlepsza znana wartość kolejnego stanu (*greedy*)

Tablica Q ma rozmiar 48×4 (stany $\times$ akcje) i jest inicjalizowana zerami. Agent korzysta ze strategii ε-zachłannej (ε-greedy): z prawdopodobieństwem ε wykonuje losowaną akcję (eksploracja), a z prawdopodobieństwem $1 - \varepsilon$ wybiera akcję o najwyższej wartości Q (eksploatacja).

3 Implementacja

Kod składa się z dwóch plików:

- `q_learning_agent.py` – klasa `QLearningAgent` zawierająca tabelę Q oraz logikę wyboru akcji i aktualizacji
- `cliff_walking.ipynb` – notatnik z pętlą trenowania i wizualizacją wyników

Główna pętla uczenia

Funkcja `train_agent()` enkapsuluje całą pętlę RL. W każdym epizodzie:

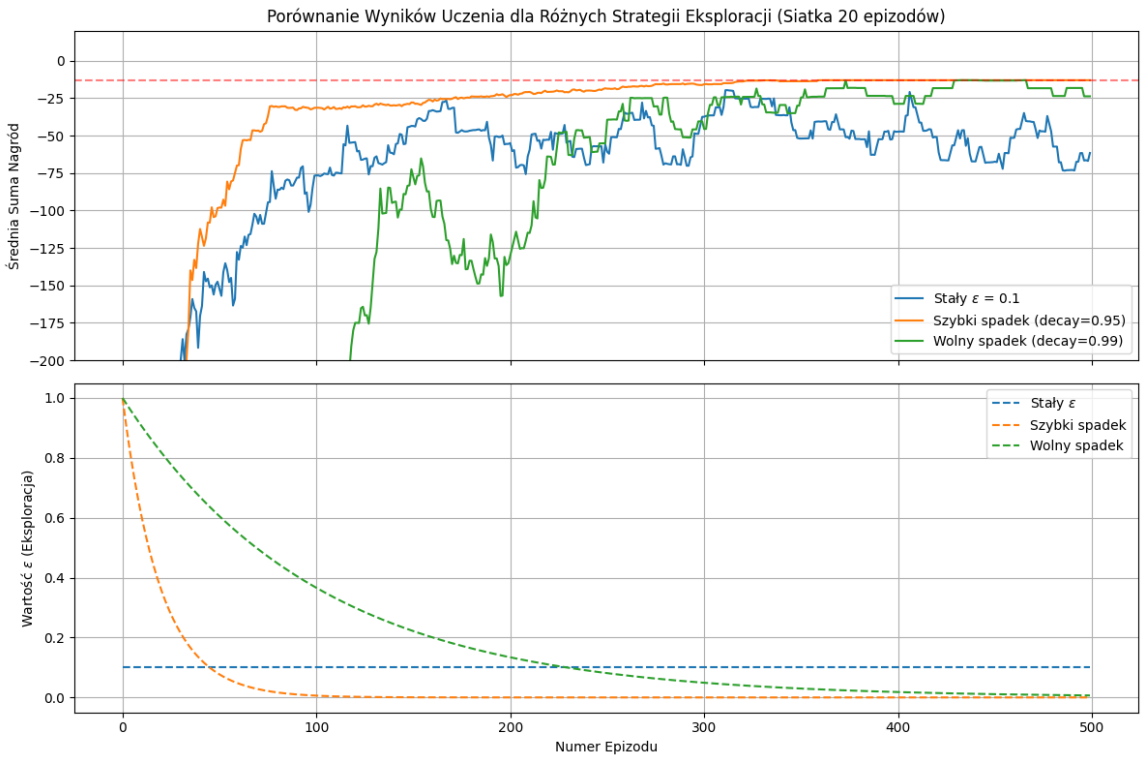
1. agent obserwuje stan s i wybiera akcję a (strategia ε-greedy)
2. środowisko zwraca nagrodę r oraz kolejny stan s'
3. tablica Q jest aktualizowana równaniem Bellmana
4. po epizodzie epsilon jest mnożony przez współczynnik wygaszania (*decay*)

Wartość terminacji przekazywana do równania aktualizacji rozróżnia stan końcowy (wytrenowany, bez przyszłych nagród) od przerwania przez limit kroków (*truncated*), co zapewnia poprawną propagację wartości Q.

4 Eksperymenty

Przeprowadzono eksperyment porównujący trzy strategie wygaszania parametru ε na przestrzeni 500 epizodów. Wszystkie pozostałe hiperparametry ($\alpha = 0.1$, $\gamma = 0.9$) pozostawały stałe. Na wykresach widoczna jest średnia krocząca z okna 20 epizodów.

Konfiguracja	**start ε**	**decay**	**Wynik (ostatnie 50 ep.)**
Stały $\varepsilon = 0.1$	0.1	1.0 (brak)	~ -40 do -60 (duża wariancja)
Szybkie wygaszanie	1.0	0.95	~ -13 (stabilnie optymalna)
Wolne wygaszanie	1.0	0.99	~ -13 do -20 (wolniejsza konwergencja)



Rysunek 2: Porównanie krzywych uczenia i przebiegu wygaszania ε dla trzech strategii eksploracji w CliffWalking.

5 Opis procesu uczenia

Faza eksploracji (epizody 0–100)

Na początku procesu uczenia agent działa niemal losowo – wysoka wartość ε powoduje częste spadki z urwiska i nagrody rzędu -100. W tym czasie tablica Q jest intensywnie zapełniana wstępnymi oszacowaniami. Krzywa uczenia wykazuje dużą wariancję.

Faza przejściowa (epizody 100–300)

W miarę wygaszania ε agent coraz częściej korzysta z wyuczonej tablicy Q. Wyniki stabilizują się i wyraźnie rosną. Szybkie wygaszanie (`decay=0.95`) wyprzedza wolniejsze –

agent wcześniej przechodzi do eksploatacji poznanej ścieżki.

Faza eksploatacji (epizody 300–500)

Po osiągnięciu $\varepsilon \approx 0$ obie wersje z wygaszaniem osiągają wynik bliski optimum (-13 pkt). Strategia ze stałym $\varepsilon = 0.1$ nigdy nie osiąga pełnej stabilności ze względu na 10% losowości.

6 Wnioski

- Q-Learning skutecznie rozwiązał problem CliffWalking, osiągając nagrodę -13.
- Strategia wygaszania ε ma krytyczny wpływ na jakość uczenia.
- Współczynnik $\gamma = 0.9$ pozwolił na poprawne zaplanowanie ścieżki omijającej urwisko.

Część II
Środowisko ciągłe: CartPole-v1

7 Opis środowiska i cel

Środowisko `CartPole-v1` z biblioteki Gymnasium modeluje klasyczny problem sterowania odwróconym wahadłem. Na wózku poruszającym się wzdłuż poziomego toru osadzony jest kij, który agent musi utrzymać w pionie — jedyną dostępną akcją jest pchnięcie wózka w lewo lub w prawo.

Parametr	**Wartość**
Przestrzeń obserwacji	Box(4,) – 4 zmienne rzeczywiste (ciągła)
Przestrzeń akcji	Discrete(2) – pchnięcie w lewo lub w prawo
Nagroda za krok	+1 pkt (każdy krok utrzymania kija)
Koniec epizodu	kąt kija > 12 lub pozycja wózka > 2,4
Wynik maksymalny	500 pkt

Tabela 1: Parametry środowiska CartPole-v1.

Rysunek 3: Środowisko CartPole-v1.

Kluczowa różnica względem `CliffWalking`: przestrzeń obserwacji jest **ciągła**. Wektor stanu zawiera cztery liczby rzeczywiste: pozycję wózka, jego prędkość, kąt kija i prędkość kątową kija. Klasyczny Q-Learning z tablicą wymaga skończonej przestrzeni stanów — konieczna jest zatem *dyskretyzacja* lub zastosowanie głębokiej sieci neuronowej.

W tej części sprawozdania prezentujemy oba podejścia:

1. **Q-Learning z dyskretyzacją** — punkt odniesienia; pokazuje ograniczenia podejścia tablicowego dla ciągłych przestrzeni obserwacji.

2. **Deep Q-Network (DQN)** — sieć neuronowa aproksymuje funkcję $Q(s, a)$ bezpośrednio na ciągłym wektorze stanu.

8 Algorytm 1: Q-Learning z dyskretyzacją

8.1 Idea dyskretyzacji

Każda z czterech zmiennych obserwacji jest mapowana na dyskretne kubełki (*bins*) przy pomocy funkcji `np.digitize`. Przyjęto 6 progów na wymiar, co tworzy 7 kubełków i daje $7^4 = 2401$ możliwych stanów dyskretnych.

Zmienna	**Zakres**	**Liczba progów**
Pozycja wózka	$[-4{,}8;\ 4{,}8]$	6
Prędkość wózka	$[-4;\ 4]$	6
Kąt kija (rad)	$[-0{,}418;\ 0{,}418]$	6
Prędkość kątowa kija	$[-4;\ 4]$	6

Tabela 2: Parametry dyskretyzacji przestrzeni obserwacji.

Tablica Q ma rozmiar 2401×2 i jest inicjalizowana zerami. Agent korzysta ze strategii ε-zachłannej identycznej jak w części pierwszej sprawozdania (CliffWalking), z hiperparametrami: $\alpha = 0{,}1$, $\varepsilon_0 = 1{,}0$, $decay = 0{,}995$, $\varepsilon_{\min} = 0{,}01$.

8.2 Eksperyment: wpływ współczynnika dyskontowego γ

Przeprowadzono trening dla trzech wartości $\gamma \in \{0{,}5;\ 0{,}9;\ 0{,}99\}$ przez 1500 epizodów. Metrykę porównawalności stanowi całkowita zdyskontowana nagroda z pierwszych 1000 epizodów przy $\gamma = 0{,}9$:

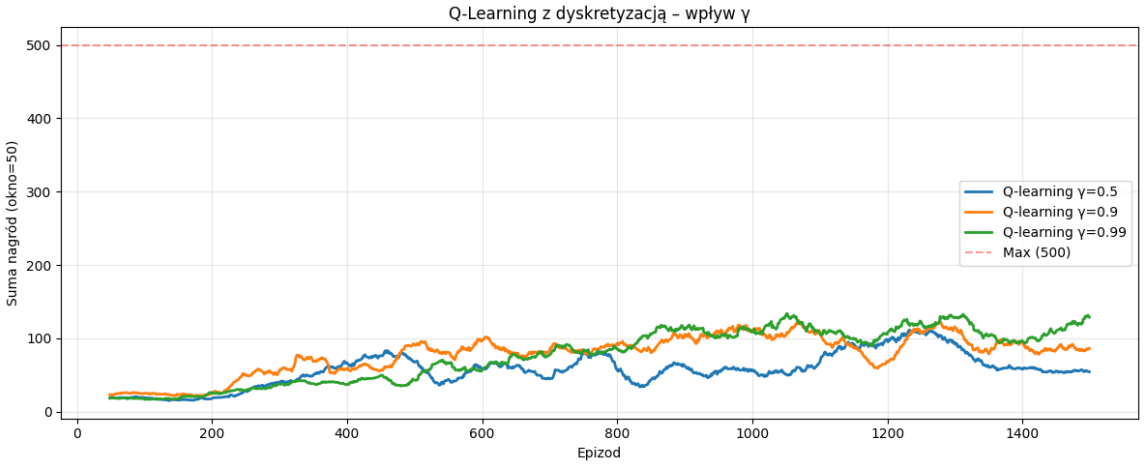
$$G = \sum_{i=0}^{999} \gamma^i \cdot R_i, \quad \gamma = 0{,}9$$

γ	**Wynik (ost. 100 ep.)**	**Zdysk. nagroda** G	**Charakterystyka**
0,5	$\sim 80-130$ pkt	209	duża wariancja, niestabilna konwergencja
0,9	$\sim 90-120$ pkt	**229**	najgładszy wzrost, najwyższe G
0,99	$\sim 100-140$ pkt	182	wysokie szczyty, niestabilna propagacja

Tabela 3: Wyniki Q-Learningu z dyskretyzacją dla trzech wartości γ. Zdyskontowana nagroda G liczona z pierwszych 1000 epizodów przy $\gamma = 0{,}9$.

8.3 Opis procesu uczenia i interpretacja γ

W fazie eksploracji (epizody 0–400) wszystkie konfiguracje zachowują się niemal identycznie — wysoki ε powoduje głównie losowe akcje i nagrody rzędu 20–40 pkt. Różnice ujawniają się po epizodzie 400, gdy ε opada poniżej 0,1.



Rysunek 4: Krzywe uczenia Q-Learningu z dyskretyzacją (średnia krocząca, okno 50 epizodów) dla trzech wartości γ. Linia przerywana oznacza maksymalny wynik 500 pkt.

- $\gamma = 0{,}5$ **(myślenie krótkoterminowe):** Agent silnie premiuje nagrody z najbliższego kroku. Reaguje szybko na chwilowe zagrożenia, lecz nie buduje strategii długofalowej. Skutkuje to dużą wariancją i brakiem stabilnego optimum — mimo że lokalne reakcje są poprawne, agent nie potrafi zaplanować utrzymania kija przez dłuższy czas. Zdyskontowana nagroda $G = 209$ jest najniższa spośród konfiguracji z wygaszaniem.

- $\gamma = 0{,}9$ **(wymagany projektem):** Balans między nagrodami natychmiastowymi a odległymi. Daje najgładszą krzywą uczenia i najwyższą zdyskontowaną nagrodę $G = 229$. W połączeniu z dyskretyzacją jest to najstabilniejsza konfiguracja — agent konsekwentnie poprawia wyuczoną politykę bez wyraźnych regresji.

- $\gamma = 0{,}99$ **(planowanie długoterminowe):** Teoretycznie powinno sprzyjać utrzymaniu kija przez 500 kroków, jednak przy niedokładnej dyskretyzacji błędy w szacowaniu odległych stanów kumulują się podczas propagacji wstecznej. Aktualizacje Bellmana stają się niestabilne — stąd pomimo chwilowych szczytów powyżej 140 pkt, zdyskontowana nagroda $G = 182$ jest *najniższa* ze wszystkich trzech wartości.

8.4 Ograniczenia dyskretyzacji

Po epizodzie 900 wszystkie konfiguracje osiągają plateau w przedziale 100–140 pkt, niezależnie od wartości γ. Wynik ten nie pochodzi z błędu implementacji — jest **sufitem dyskretyzacji**: przy 7 kubełkach na wymiar zbyt wiele różnych stanów fizycznych trafia do tego samego indeksu tablicy Q, uniemożliwiając dalsze rozróżnianie akcji. Zwiększenie liczby kubełków poprawia dokładność, ale wykładniczo zwiększa liczbę stanów do odwiedzenia (N^4 dla N kubełków), wydłużając czas eksploracji ponad praktyczne możliwości.

Rozwiązaniem jest bezpośrednia aproksymacja $Q(s, a)$ siecią neuronową, co eliminuje konieczność dyskretyzacji.

9 Algorytm 2: Deep Q-Network (DQN)

9.1 Idea i kluczowe mechanizmy

DQN zastępuje tablicę Q siecią neuronową, która dla dowolnego ciągłego wektora stanu s zwraca wartości $Q(s, a)$ dla każdej akcji. Implementacja opiera się na oficjalnym tutorialu PyTorch i zawiera dwa mechanizmy stabilizujące uczenie:

1. **Experience Replay:** przejścia (s, a, r, s') są zapisywane do bufora (*Replay Memory*) o pojemności 10 000 i losowane w mini-batchach po 256 próbek. Przerywa to korelację czasową między kolejnymi aktualizacjami.

2. **Target Network z soft update:** zamiast kopiowania wag co N kroków (hard update), target network jest aktualizowana co krok według reguły:

$$\theta_{\text{target}} \leftarrow \tau \cdot \theta_{\text{policy}} + (1 - \tau) \cdot \theta_{\text{target}}, \quad \tau = 0{,}01$$

   Wagi target network zmieniają się *płynnie*, co eliminuje skokowe destabilizacje charakterystyczne dla hard update.

Jako funkcję straty użyto Huber loss (`SmoothL1Loss`), a gradienty są przycinane do wartości bezwzględnej 100 (*gradient clipping*).

9.2 Dwie architektury sieci

Porównano dwie konfiguracje architektury sieci:

Architektura	**Warstwy**	**Parametry**
`DQN_128x128`	$4 \to 128 \to 128 \to 2$	18 818
`DQN_128x64`	$4 \to 128 \to 64 \to 2$	9 474

Tabela 4: Porównane architektury sieci neuronowej. Aktywacja ReLU po każdej warstwie ukrytej. Optymalizator AdamW z `amsgrad=True`.

9.3 Przeszukiwanie hiperparametrów

Przetestowano 3 konfiguracje hiperparametrów dla każdej architektury (łącznie 6 treningów po 600 epizodów). Zmieniano jeden parametr względem konfiguracji bazowej, aby ocenić jego izolowany wpływ.

Konfiguracja	**Zmiana**	**BATCH**	**LR**	**EPS_DECAY**
bazowa	—	256	$5 \cdot 10^{-4}$	500
wolniejszy decay	dłuższa eksploracja	256	$5 \cdot 10^{-4}$	1000
niższe LR	ostrożniejsze aktualizacje	256	$1 \cdot 10^{-4}$	500

Tabela 5: Siatka hiperparametrów. Pozostałe parametry stałe: $\gamma = 0{,}99$, $\varepsilon_0 = 0{,}9$, $\varepsilon_{\min} = 0{,}05$, TAU $= 0{,}01$.

Metryka optymalizacji: całkowita zdyskontowana nagroda z 600 epizodów przy $\gamma = 0{,}99$:

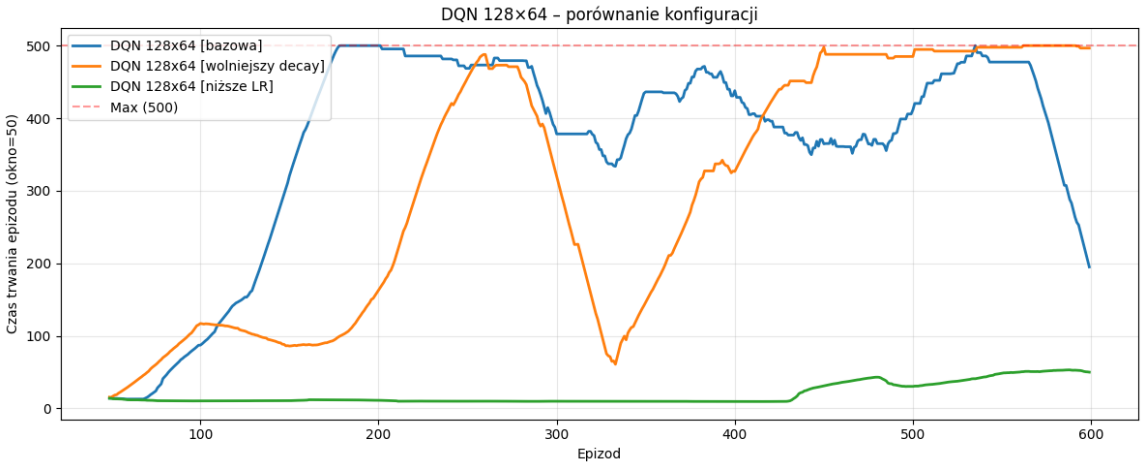
$$G_{\text{DQN}} = \sum_{i=0}^{599} \gamma^i \cdot D_i, \quad \gamma = 0{,}99$$

gdzie D_i to czas trwania i-tego epizodu (równy sumie nagród, bo nagroda $= +1$ co krok).

9.4 Wyniki przeszukiwania hiperparametrów

Architektura	Konfiguracja	Zdysk. nagroda G_{DQN}
`DQN_128x128`	bazowa	**17 902**
`DQN_128x64`	bazowa	16 747
`DQN_128x128`	wolniejszy decay	16 317
`DQN_128x64`	wolniejszy decay	10 725
`DQN_128x128`	niższe LR	2 046
`DQN_128x64`	niższe LR	1 221

Tabela 6: Ranking konfiguracji DQN według zdyskontowanej nagrody G_{DQN} z 600 epizodów ($\gamma = 0{,}99$). Najlepsza konfiguracja: `DQN_128x128` bazowa.



Rysunek 5: Krzywe uczenia dla architektury `DQN_128x64` — porównanie trzech konfiguracji hiperparametrów.

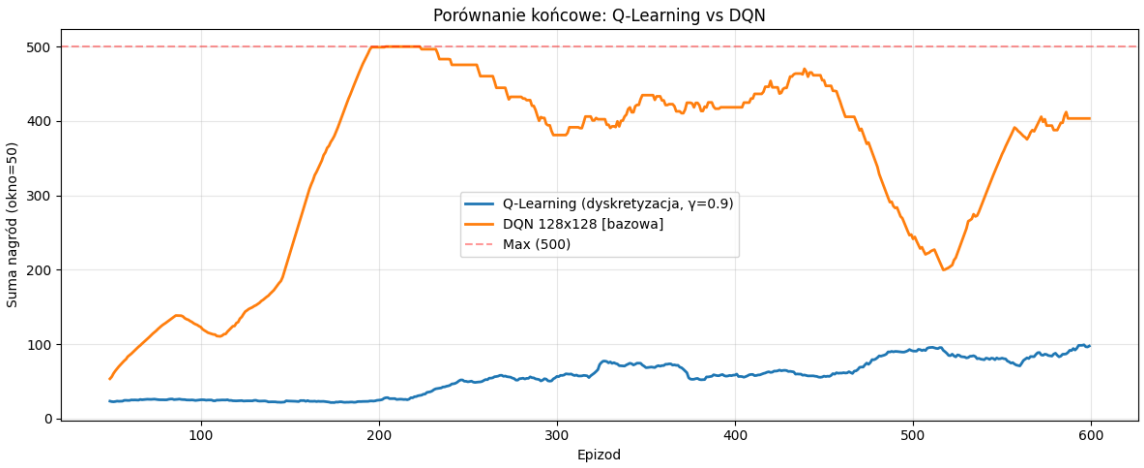
9.5 Interpretacja wyników

Wyniki wykazują wyraźny wzorzec:

- **Konfiguracja bazowa ($G \approx 17\,900/16\,700$):** Domyślne parametry z tutoriala PyTorch okazały się najlepsze. ε opada do minimum po ≈ 1500 krokach (kilkanaście epizodów), co pozwala agentowi szybko przejść do eksploatacji wyuczonej polityki.
- **Wolniejszy decay ($G \approx 16\,300/10\,700$):** Wydłużenie eksploracji do ≈ 3000 kroków (EPS_DECAY=1000) pozwala agentowi dokładniej poznać przestrzeń stanów, ale opóźnia konwergencję. Dla `DQN_128x64` różnica jest znaczna (-36%), co sugeruje że mniejsza sieć potrzebuje mniej danych do nauki stabilnej polityki.

- **Niższe LR** ($G \approx 2\,046/1\,221$)**:** Obniżenie LR z $5 \cdot 10^{-4}$ do $1 \cdot 10^{-4}$ drastycznie pogarsza wyniki — przy 600 epizodach sieć nie zdąża nauczyć się użytecznej polityki. Mechanizm soft update z $\tau = 0{,}01$ wymaga stosunkowo wysokiego LR, aby target network nadążał za poprawiającą się siecią główną.

10 Porównanie algorytmów: Q-Learning vs DQN



Rysunek 6: Porównanie końcowe: Q-Learning z dyskretyzacją ($\gamma = 0{,}9$) vs najlepsza konfiguracja DQN (`DQN_128x128`, bazowa, $\gamma = 0{,}99$). Średnia krocząca z okna 50 epizodów.

Algorytm	**Repr. stanów**	**Wynik (ost. 100 ep.)**	G **(1000 ep.)**
Q-Learning, $\gamma = 0{,}5$	dyskretna	80−130 pkt	209
Q-Learning, $\gamma = 0{,}9$	dyskretna	90−120 pkt	**229**
Q-Learning, $\gamma = 0{,}99$	dyskretna	100−140 pkt	182
DQN 128x128 [bazowa] $\gamma = 0{,}99$	ciągła	≈ 500 pkt	2*$\sim 17\,900$

Tabela 7: Zbiorcze porównanie algorytmów. G dla Q-Learningu liczone przy $\gamma = 0{,}9$ z 1000 epizodów; dla DQN przy $\gamma = 0{,}99$ z 600 epizodów.

Różnica między algorytmami jest fundamentalna:

- Q-Learning z dyskretyzacją utyka na plateau ~ 140 pkt (28% maksimum) niezależnie od γ i liczby epizodów. Jest to ograniczenie reprezentacji, nie algorytmu.
- DQN osiąga wynik ≈ 500 pkt (maksimum środowiska) po kilkuset epizodach. Sieć neuronowa aproksymuje $Q(s, a)$ na ciągłym wektorze stanu bez utraty informacji.
- Kluczowym ulepszeniem DQN względem naiwnego Q-Learningu z siecią jest **soft update** target network ($\tau = 0{,}01$ co krok). W porównaniu z hard update co N kroków, wagi target network zmieniają się płynnie — eliminuje to skokowe destabilizacje, które prowadziły do *catastrophic forgetting* (gwałtownego spadku wyniku z ~ 450 do ~ 60 pkt obserwowanego przy hard update).

11 Wizualizacja wytrenowanego agenta

Po zakończeniu treningu najlepsza sieć (`DQN_128x128`, konfiguracja bazowa) jest zapisywana do pliku `best_policy_net.pt`. Plik `cartpole_visualize.py` ładuje zapisany model i uruchamia animację w czasie rzeczywistym przy użyciu biblioteki `pygame`.

Agent działa czysto zachłannie — bez eksploracji ($\varepsilon = 0$). Wyświetlane są: aktualny kąt kija, pozycja wózka, numer kroku oraz suma nagród. Sterowanie: klawisz **R** resetuje epizod, **Q** kończy wizualizację.

12 Wnioski

1. **Dyskretyzacja tworzy sufit.** Q-Learning z tablicą osiąga $\sim 100-140$ pkt (20–28% maksimum) niezależnie od γ. Sufit wynika z utraty informacji przy mapowaniu stanów na kubełki — nie jest błędem implementacji.

2. **Wpływ γ na Q-Learning.** $\gamma = 0{,}9$ daje najwyższą zdyskontowaną nagrodę ($G = 229$) i najstabilniejsze uczenie. $\gamma = 0{,}99$ jest kontr-produktywne przy dyskretyzacji: błędy kubełkowania kumulują się przy propagacji długoterminowych wartości Q, obniżając G do 182. $\gamma = 0{,}5$ daje dużą wariancję i pośrednią wartość $G = 209$.

3. **DQN eliminuje ograniczenia dyskretyzacji.** Najlepsza konfiguracja (`DQN_128x128`, bazowa) osiąga zdyskontowaną nagrodę $G_{\text{DQN}} = 17\,902$ — o rzędy wielkości wyższą niż Q-Learning — i regularnie osiąga maksymalny wynik 500 pkt.

4. **Soft update jest kluczowy dla stabilności.** $\tau = 0{,}01$ co krok eliminuje catastrophic forgetting. Przy hard update obserwowano gwałtowne restarty uczenia; przy soft update krzywa rośnie monotonicznie.

5. **LR ma dominujący wpływ na wyniki DQN.** Obniżenie LR z $5 \cdot 10^{-4}$ do $1 \cdot 10^{-4}$ redukuje G o $\sim 89\%$ dla obu architektur — jest to najważniejszy hiperparametr przy 600 epizodach treningu. Prędkość eksploracji (EPS_DECAY) ma umiarkowany wpływ.

6. **Architektura sieci ma drugorzędne znaczenie dla CartPole.** `DQN_128x128` ($G = 17\,902$) nieznacznie wyprzedza `DQN_128x64` ($G = 16\,747$) przy konfiguracji bazowej. Różnica wynosi $\sim 7\%$ — CartPole jest wystarczająco proste, by obie architektury skutecznie je rozwiązywały.